

Computational Approaches to Biological Challenges (algorithmic primer)

Shortest Common Superstring & Lander-
Waterman Statistics

What is Computer Science?

Not actually simple to define constructively

Still debate whether certain areas constitute CS

Computer science is the scientific and practical approach to computation and its applications. It is the systematic study of the feasibility, structure, expression, and mechanization of the methodical procedures (or algorithms) that underlie the acquisition, representation, processing, storage, communication of, and access to information* ...

*<http://www.cs.bu.edu/AboutCS/WhatIsCS.pdf>

What is Computer Science?

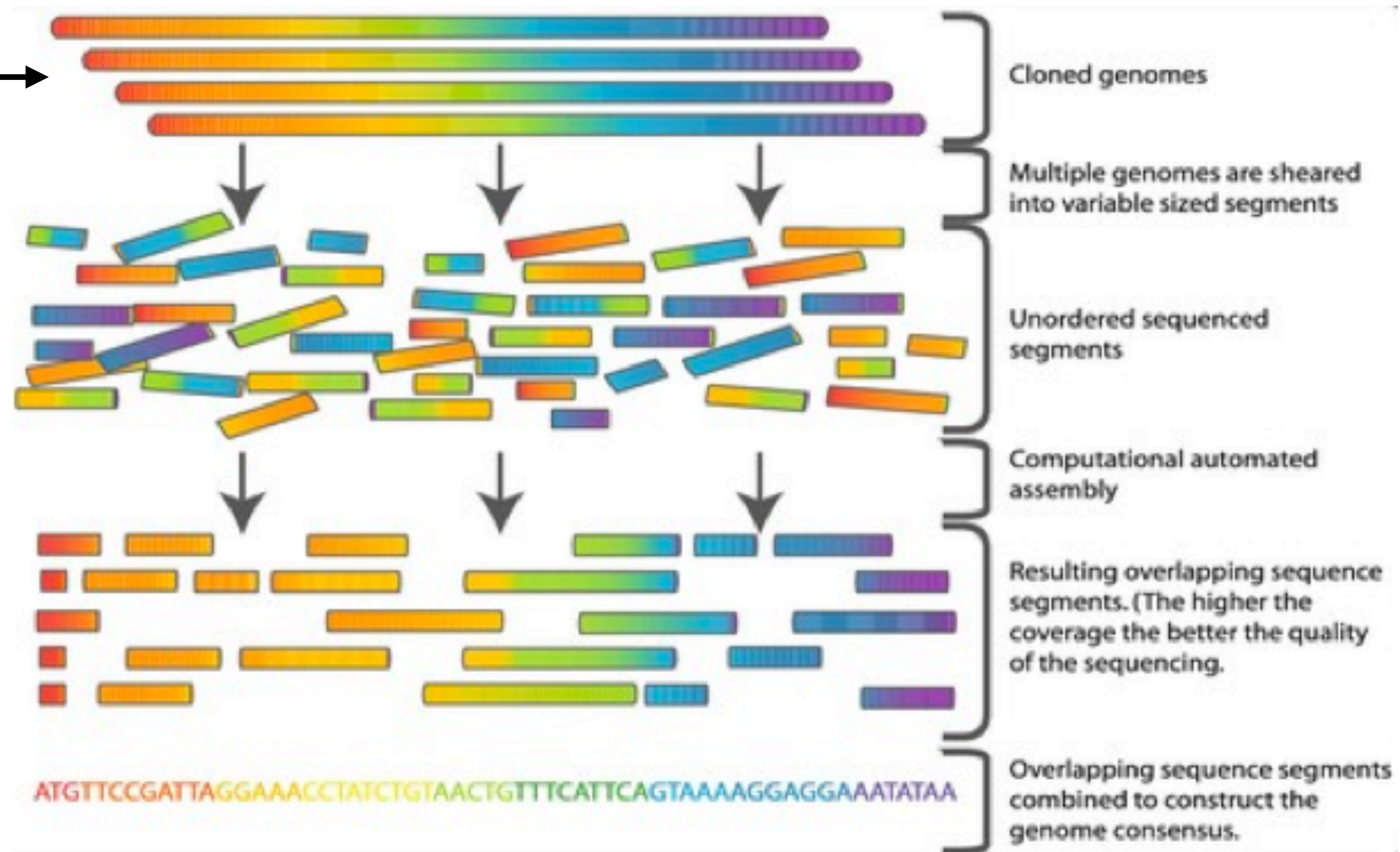
Concerned with the development of provably **correct** and **efficient** computational procedures (algorithms & data structures) to answer **well-specified** problems.

To answer a computational question, we first need a well-formulated problem.

It turns out that a **major challenge** in bioinformatics will simply be determining how to frame the *computational problem* corresponding to a *biological question* in a well-posed and meaningful way!

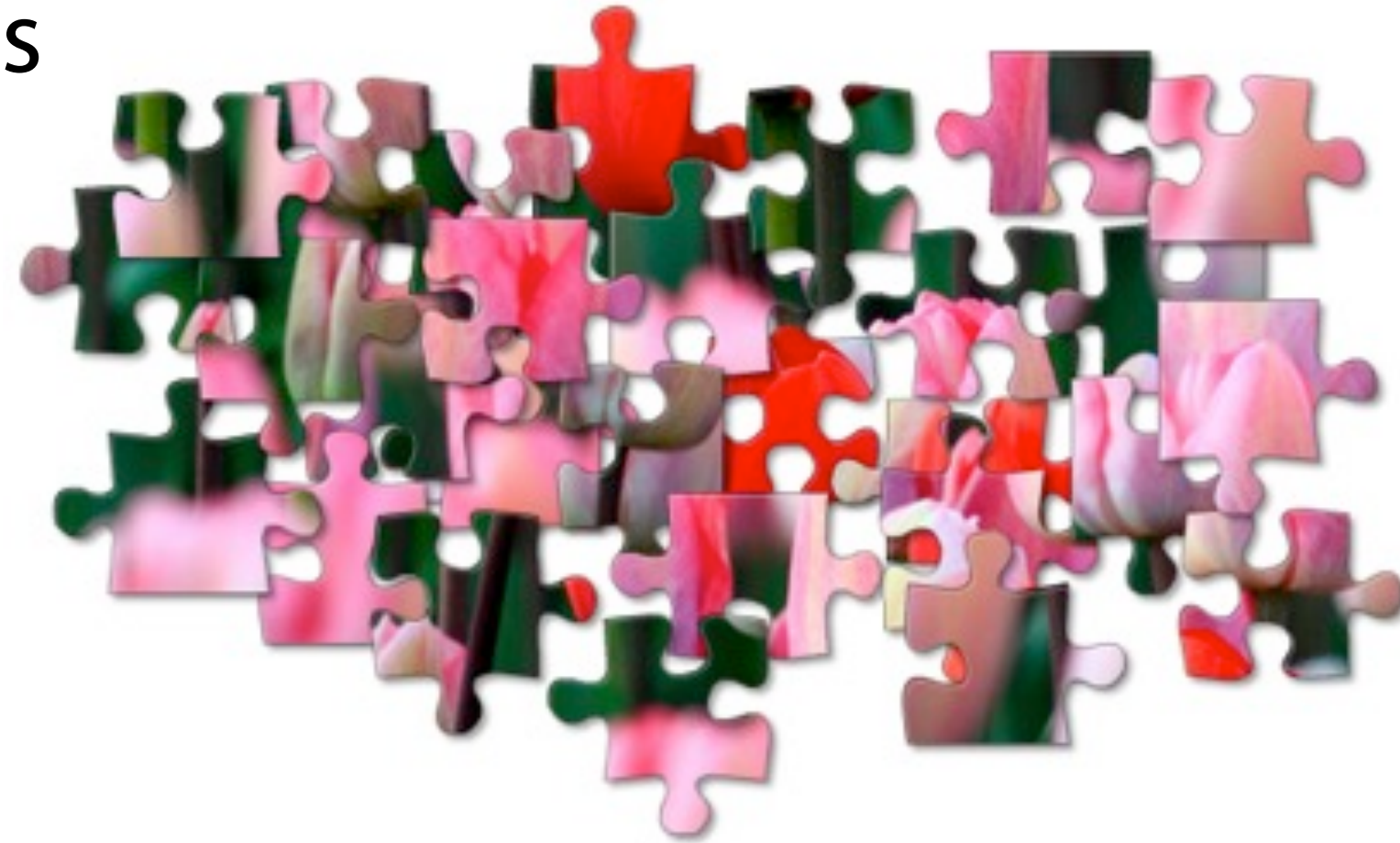
What is genome assembly: intuitively?

Why start with
many cloned genomes →
and not just one?



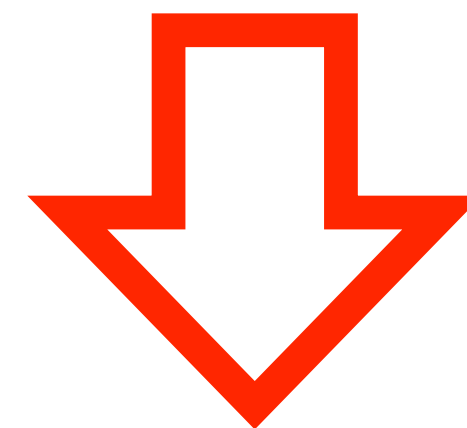
Assembly

Reads



+

Reference genome



Input DNA



How to assemble
puzzle without the
benefit of knowing
what the finished
product looks like?

Assembly

Whole-genome “shotgun” sequencing starts by copying and fragmenting the DNA

(“Shotgun” refers to the random fragmentation of the whole genome; like it was fired from a shotgun)

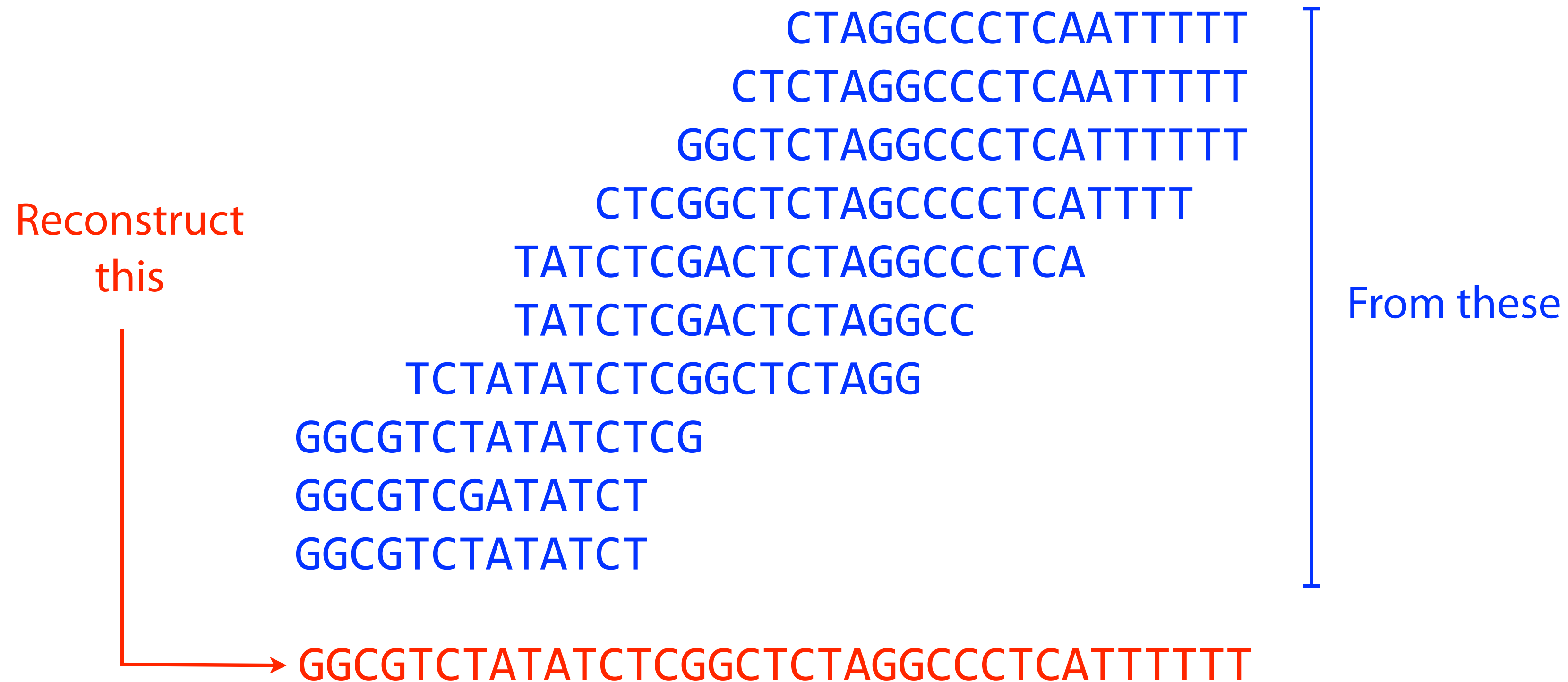
Input: GGCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT

Copy: GGCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT
GGCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT
GGCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT
GGCGTCTATATCTCGGCTCTAGGCCCTCATTTTTTT

Fragment: GGCGTCTA TATCTCGG CTCTAGGCCCTC ATTTTTTT
GGC GTCTATAT CTCGGCTCTAGGCCCTCA TTTTTT
GGCGTC TATATCT CGGCTCTAGGCCCT CATTTTTTT
GGCGTCTAT ATCTCGGCTCTAG GCCCTCA TTTTTT

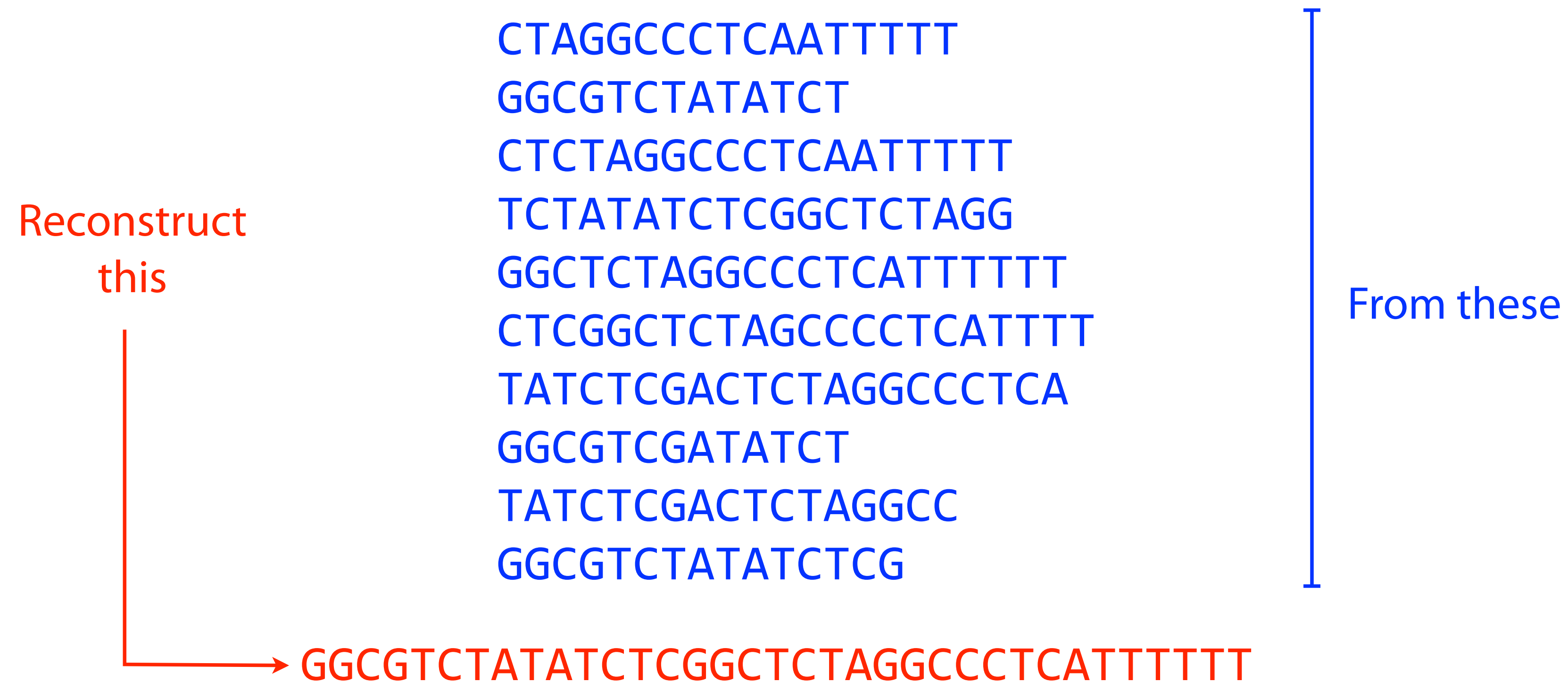
Assembly

Assume sequencing produces such a large # fragments that almost all genome positions are *covered* by many fragments...



Assembly

...but we don't know what came from where



Assembly



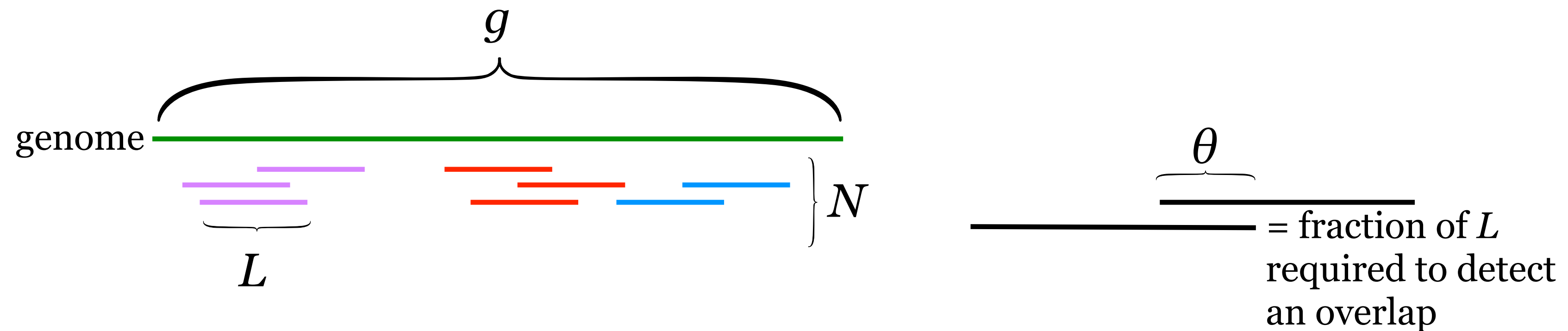
In general: we don't even know exactly how *long* the original string was!

Aside: How Much Coverage is Enough?

Lander-Waterman Statistics

Lander ES, Waterman MS (1988). "Genomic mapping by fingerprinting random clones: a mathematical analysis". Genomics 2 (3): 231–239

How many reads do we need to be sure we cover the whole genome?



An **island** is a contiguous group of reads that are connected by overlaps of length $\geq \theta L$.
(Various colors above)

Want: Expression for expected # of islands given N, g, L, θ .

Expected # of Islands

$\lambda := N/g$ = expected # of read starts per position
 (a rate, not a probability; random sampling)

Pr(k reads start in an interval of length x)

x trials, want k “successes”, small probability λ of success

Expected # of successes = λx

Poisson approximation to binomial distribution:

$$\Pr(k \text{ reads in length } x) = e^{-\lambda x} \frac{(\lambda x)^k}{k!}$$

Expected # of islands = $N \times \Pr(\text{read is at rightmost end of island})$

$$\begin{aligned} \frac{\overbrace{(1-\theta)L} \quad \theta L}{\text{—————}} &= N \times \Pr(0 \text{ reads start in } (1-\theta)L) \\ &= N e^{-\lambda(1-\theta)L} \frac{(\lambda x)^0}{0!} \quad (\text{from above}) \\ &= N e^{-\lambda(1-\theta)L} \\ &= N e^{-(1-\theta)LN/g} \quad \leftarrow LN/g \text{ is called the } \mathbf{coverage} \mathbf{ } c. \end{aligned}$$

Expected # of Islands, 2

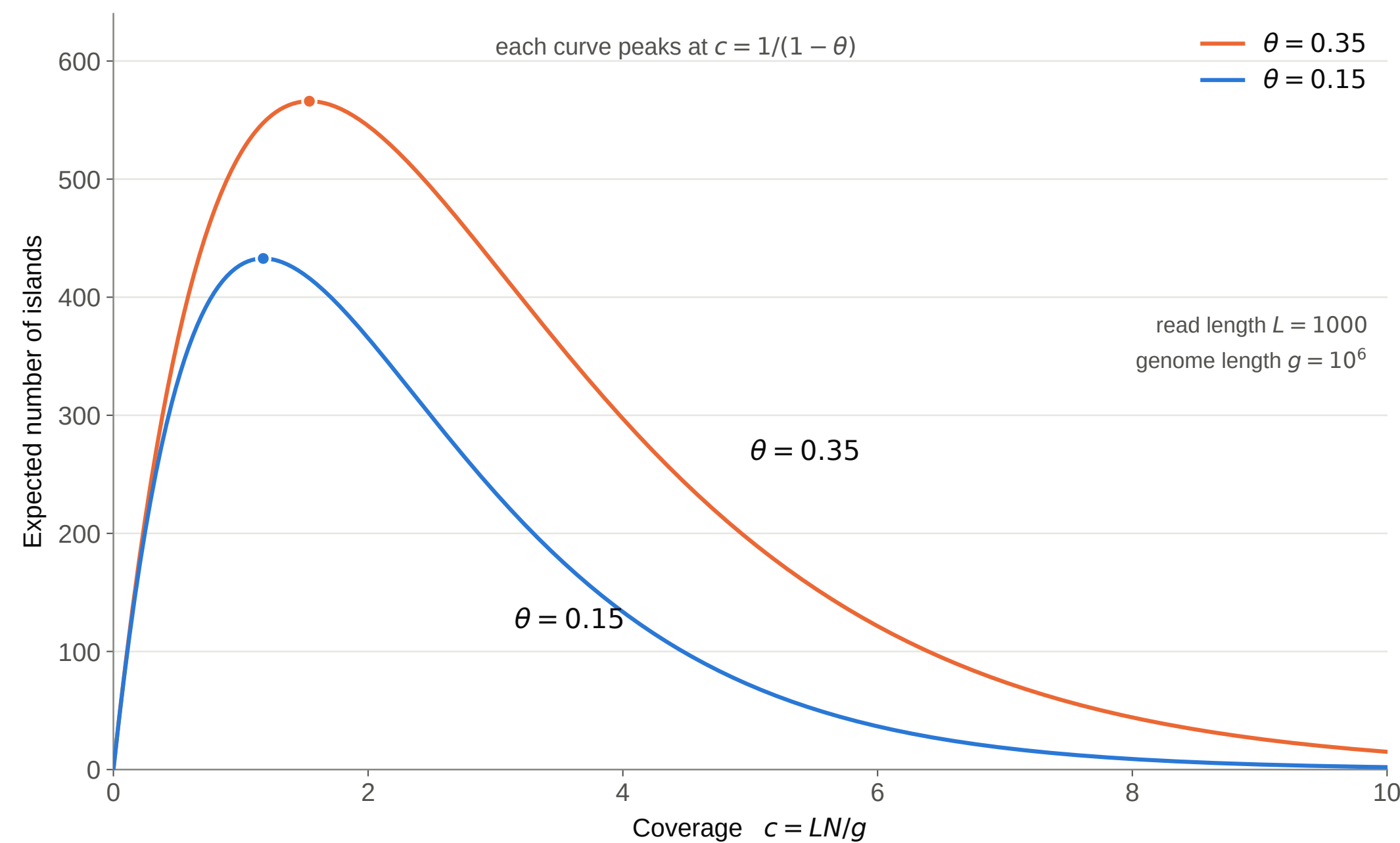
We can rewrite this expression to depend more directly on the things we can control: c and θ

$$\text{Expected \# of islands} = N e^{-(1-\theta) L N / g}$$

$$= N e^{-(1-\theta) c}$$

$$= \frac{L/g}{L/g} N e^{-(1-\theta) c}$$

$$= \frac{g}{L} c e^{-(1-\theta) c}$$



How much coverage is enough?

A base is uncovered exactly when no read starts in the L positions ending at it. Set $x = L$ in the same Poisson:

$$\Pr(\text{base uncovered}) = e^{-\lambda L} = e^{-c}$$

and therefore

$$\text{expected uncovered bases} = g e^{-c}$$

$$\text{expected number of gaps} = N e^{-c}$$

$$\text{average gap length} = g/N = L/c$$

Human genome

$$g = 3.1 \times 10^9, \quad L = 1000, \quad c = 8 \times$$

reads $N = cg/L$	24,800,000
e^{-c}	3.35×10^{-4}
uncovered bases	1.04 Mb
gaps	8,300
average gap	125 bp

At $8 \times$ coverage you still expect ~ 1 Mb of the genome missing, in $\sim 8,300$ separate holes.

expected uncovered bases < 1 needs

$$c > \ln g \approx 22 \times$$

Formulating *a* genome assembly problem

Concerned with the development of provably **correct** and **efficient** computational procedures (algorithms & data structures) to answer **well-specified** problems.

To answer a computational question, we first need a well-formulated problem.

Given: a collection, R , of sequencing reads (strings)

Find: The genome (string), G , that generated them

Formulating a genome assembly problem

Concerned with the development of provably **correct** and **efficient** computational procedures (algorithms & data structures) to answer **well-specified** problems.

To answer a computational question, we first need a well-formulated problem.

Given: a collection, R , of sequencing reads (strings)

Find: The genome (string), G , that generated them

Not well-specified.

What makes one genome more likely than another?

What constraints do we place on the space of solutions?

Formulating *a* genome assembly problem

Concerned with the development of provably **correct** and **efficient** computational procedures (algorithms & data structures) to answer **well-specified** problems.

To answer a computational question, we first need a well-formulated problem.

Given: a collection, R , of sequencing reads (strings)

Find: The shortest genome (string), G , that contains all of them



Shortest Common Superstring

Given: a collection, $S = \{s_1, s_2, \dots, s_k\}$, of sequencing reads (strings)

Find*: The shortest possible genome (string), G , such that $s_1, s_2, \dots, s_k$ are all substrings of G

How might we go about solving this problem?

*for reasons we'll explore later, this isn't actually a great formulation for genome assembly.

SCS, stated as a **decision** problem

SCS-DECISION

Given: a set $S = \{s_1, \dots, s_k\}$ of strings and an integer m

Question: is there a string G with $|G| \leq m$ such that every s_i is a substring of G ?

- The optimization and decision versions are polynomially equivalent — binary search on m .
- SCS-DECISION is in NP: G itself is the certificate — check $|G| \leq m$ and k substring tests, all polynomial.
- NP-hardness needs a reduction *from* a known NP-complete problem *into* SCS. Modelling SCS *as* TSP does not do this.

This is the distinction the next slides turn on: we model SCS as TSP to understand it, but that is not a proof.

Shortest common superstring

Given a collection of strings S , find $SCS(S)$: the shortest string that contains all strings in S as substrings

Without requirement of “shortest,” it’s easy: just concatenate them

Example: S : BAA AAB BBA ABA ABB BBB AAA BAB

Concatenation: BAAAABBBBAABAABBBBBBAAABAB
└────────── 24 ─────────┘

$SCS(S)$: AAABBBABAA
└── 10 ─┘

AAA
AAB
ABB
BBB
BBA
BAB
ABA
BAA

Idea: pick order for strings in *S* and construct superstring

order 1: AAA AAB ABA ABB BAA BAB BBA BBB
AAA

Idea: pick order for strings in *S* and construct superstring

order 1: AAA AAB ABA ABB BAA BAB BBA BBB
AAAB

Idea: pick order for strings in *S* and construct superstring

order 1: AAA AAB ABA ABB BAA BAB BBA BBB
 └────────┘
 AAABA

Idea: pick order for strings in *S* and construct superstring

order 1: AAA AAB ABA ABB BAA BAB BBA BBB
AAABABB

Idea: pick order for strings in *S* and construct superstring

order 1: AAA AAB ABA ABB BAA BAB BBA BBB

AAABABBAABABBABBB ← superstring 1

Idea: pick order for strings in S and construct superstring

order 1: AAA AAB ABA ABB BAA BAB BBA BBB

AAABABBAABABBABBB ← superstring 1

order 2: AAA AAB ABA BAB ABB BBB BAA BBA

AAABABBBBAABBA ← superstring 2

Try all possible orderings and pick shortest superstring

If S contains n strings, $n!$ (n factorial) orderings possible

order 1: AAA AAB ABA ABB BAA BAB BBA BBB

AAABABBAABABBABBB ← superstring 1

order 2: AAA AAB ABA BAB ABB BBB BAA BBA

AAABABBBBAABBA ← superstring 2

If S contains n strings, $n!$ (n factorial) orderings possible

Shortest common superstring

Can we solve it?

Imagine a modified overlap graph where each edge has cost = - (length of overlap)

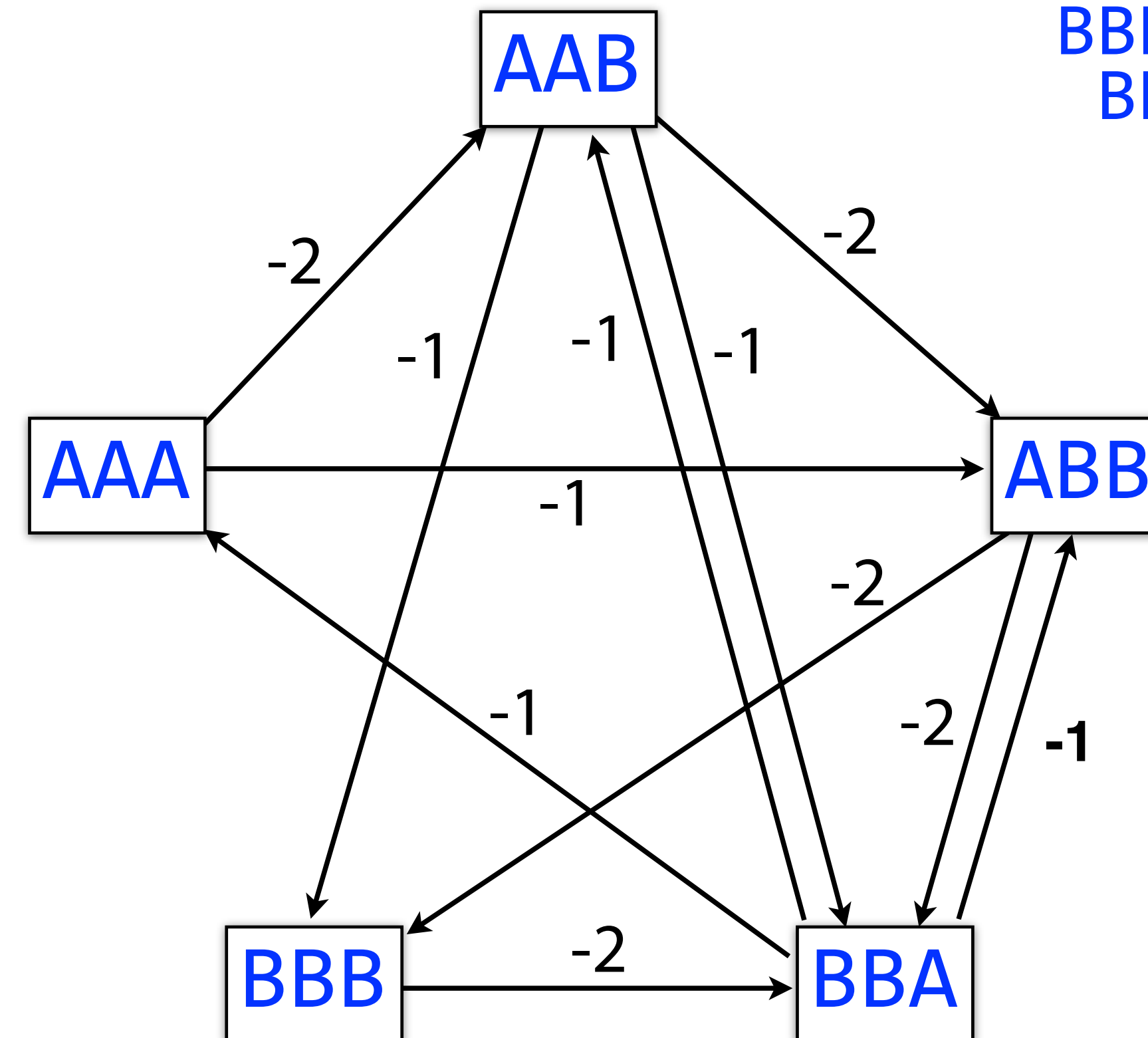
SCS corresponds to a path that visits every node once, minimizing total cost along path

That's the *Traveling Salesman Problem (TSP)*, which is NP-hard!

S: AAA AAB ABB BBB BBA

SCS(S): AAABBBBA

AAA
AAB
ABB
BBB
BBA



Shortest common superstring

Say we disregard edge weights and just look for a path that visits all the nodes exactly once

That's the *Hamiltonian Path* problem: NP-complete

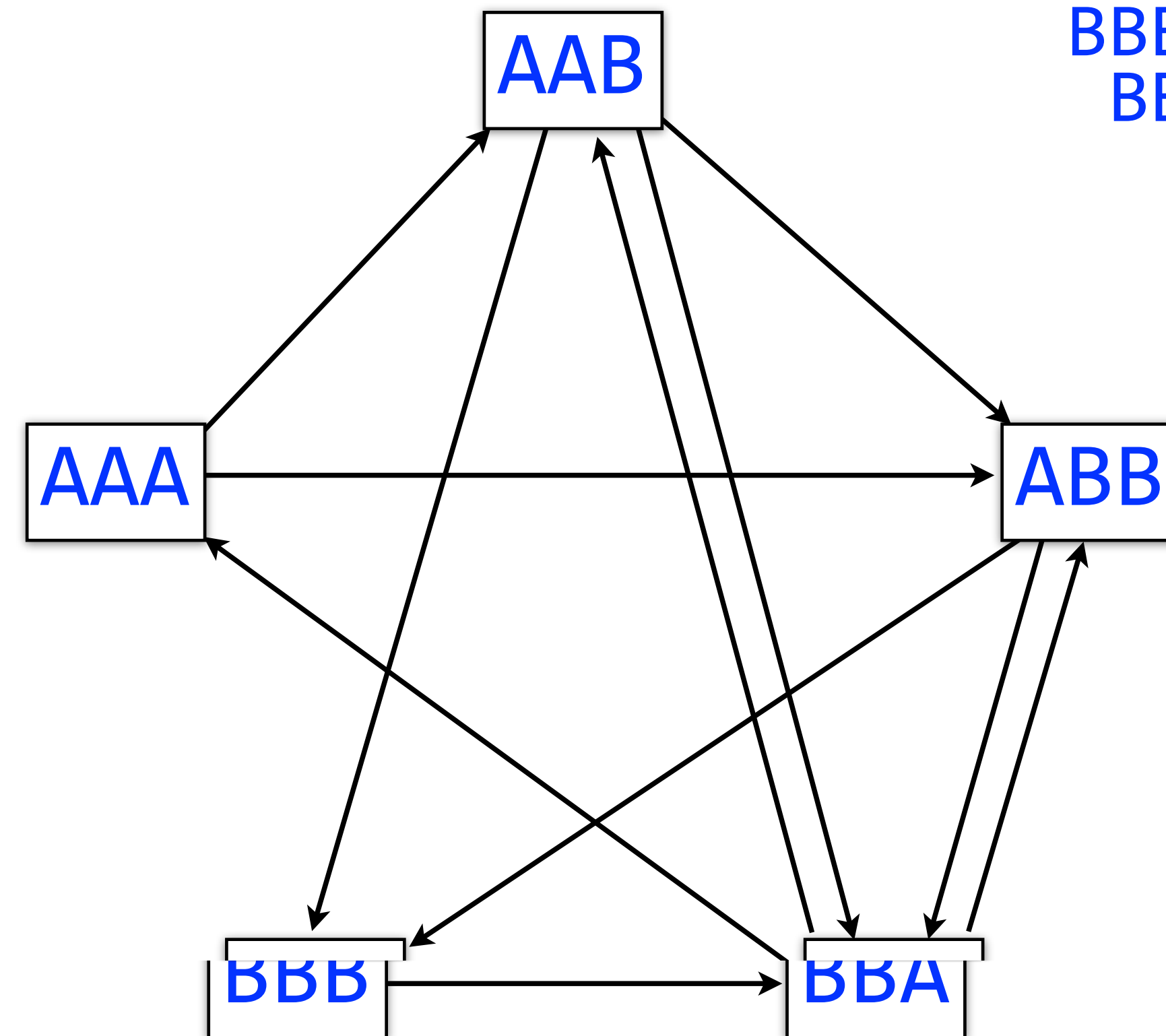
So, it's not even the weights that make visiting all nodes once hard

Indeed, it's well established that SCS is NP-hard

S : AAA AAB ABB BBB BBA

$SCS(S)$: AAABBBBA

AAA
AAB
ABB
BBB
BBA



Shortest common superstring & friends

Traveling Salesman, Hamiltonian Path, and Shortest Common Superstring are all NP-hard

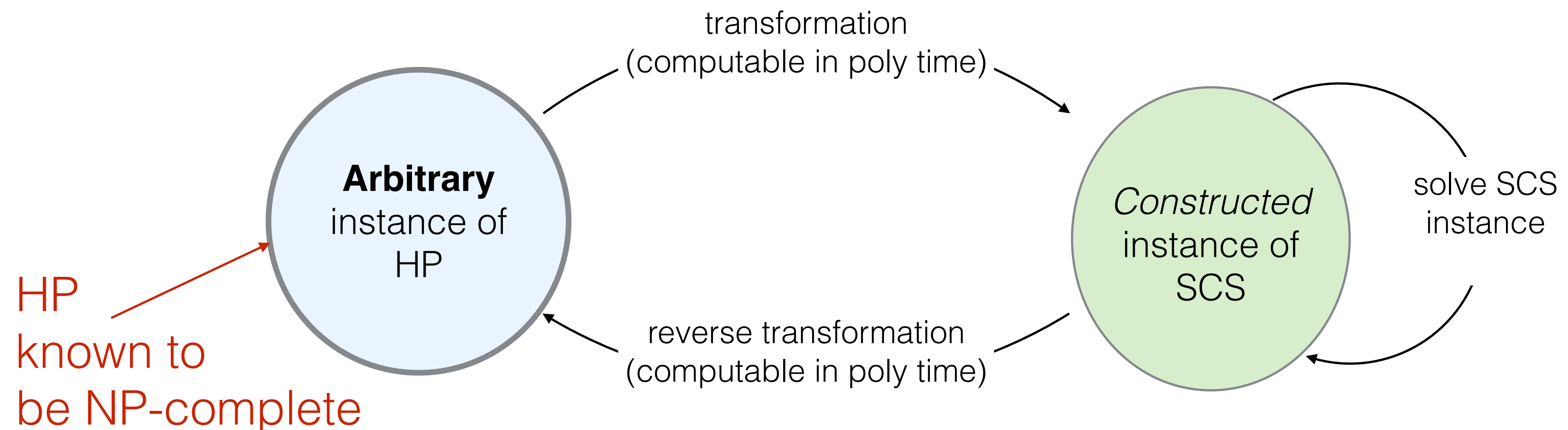
For refreshers on Traveling Salesman, Hamiltonian Path, NP-hardness and NP-completeness, see Chapters 34 and 35 of “Introduction to Algorithms” by Cormen, Leiserson, Rivest and Stein, or Chapters 8 and 9 of “Algorithms” by Dasgupta, Papadimitriou and Vazirani (free online: <http://www.cs.berkeley.edu/~vazirani/algorithms>)

Who remembers reductions from 351?

Important note: The fact that we modeled SCS as NP-hard problems (TSP and HP) **does not** prove that (the decision version of) SCS is NP-complete. To do that, we must **reduce** a known NP-complete problem to **SCS**.

Given an instance I of a known hard problem, **generate** an instance I' of SCS such that if we can solve I' in polynomial time, then we can solve I in polynomial time. This *implies* that SCS is *at least* as hard as the hard problem.

This can be done e.g. with HAMILTONIAN PATH



Shortest Common Superstring

The fact that (the decision version of) SCS is **NP-complete** means that it is unlikely that there exists *any* algorithm that can solve a general instance of this problem in time polynomial in n — the number of input strings (i.e. reads in the case of genome assembly).

If we give up on finding a *shortest* possible superstring G , and instead look for one that's “near-shortest”, how does the situation change?

Shortest Common Superstring

There's a “greedy” *heuristic* that turns out to be an *approximation algorithm* (provides a solution within a constant factor of the optimum)
Different approx. **(not all greedy)**

At each step, choose the pair of strings with the maximum overlap, merge them, and return the merged string to the collection.

Greedy conjecture factor of 2-
OPT is the worst case

Open conjecture! We can prove 3.5, but many believe the factor is actually 2.

ratio	authors	year
approximating SCS		
3	Blum, Jiang, Li, Tromp and Yannakakis [4]	1991
$2\frac{8}{9}$	Teng, Yao [23]	1993
$2\frac{5}{6}$	Czumaj, Gasieniec, Piotrow, Rytter [8]	1994
$2\frac{50}{63}$	Kosaraju, Park, Stein [15]	1994
$2\frac{3}{4}$	Armen, Stein [1]	1994
$2\frac{50}{69}$	Armen, Stein [2]	1995
$2\frac{2}{3}$	Armen, Stein [3]	1996
$2\frac{25}{42}$	Breslauer, Jiang, Jiang [5]	1997
$2\frac{1}{2}$	Sweedyk [21]	1999
$2\frac{1}{2}$	Kaplan, Lewenstein, Shafrir, Sviridenko [12]	2005
$2\frac{1}{2}$	Paluch, Elbassioni, van Zuylen [18]	2012
$2\frac{11}{23}$	Mucha [16]	2013
$2\frac{11}{30}$	Paluch	2014

Shortest Common Superstring

There's a “greedy” *heuristic* that turns out to be an *approximation algorithm* (provides a solution within a constant factor of the optimum)

At *each step*, choose the *pair of strings* with the *maximum overlap*, merge them, and return the merged string to the collection.

Greedy conjecture factor of 2-OPT *is* the worst case

Open conjecture? We can prove 3.5, but many believe the factor is actually 2 (but now that seems unlikely).

Disproving the Greedy Superstring Conjecture

Hiroki Shibata^{*1}

¹Joint Graduate School of Mathematics for Innovation, Kyushu University, Japan

September 2, 2026

Abstract

The *shortest common superstring problem* is to find the shortest string that contains every string in a given set as a substring. It is conjectured that the greedy algorithm that repeatedly selects a pair of strings with maximum overlap and merges them is a 2-approximation algorithm, and this conjecture had remained open for nearly four decades. In this paper, we disprove this conjecture and show that the approximation ratio of this algorithm is at least $9/4$.

<https://arxiv.org/pdf/2609.01365>

Overlap

a suffix of a is a prefix of b

a TATTGCA

b GCAAGGC

 ↔

 overlap = 3

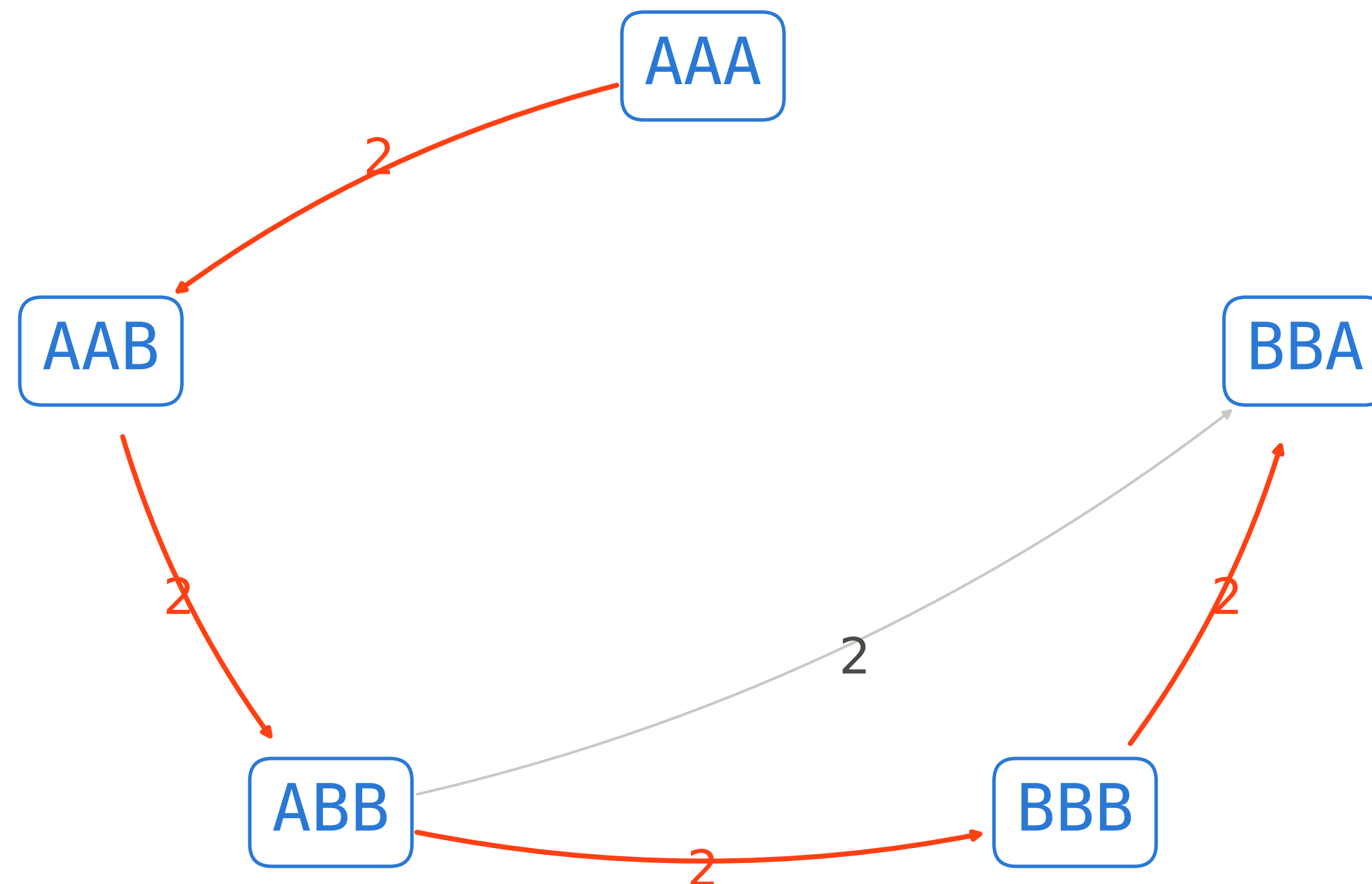
$\text{ov}(a, b)$ = length of the longest suffix of a that is a prefix of b

$\text{ov}(a, b) \neq \text{ov}(b, a)$ — overlap is *directed*

Merging a and b on an overlap of w gives a string of length $|a| + |b| - w$:
every unit of overlap is a character saved, so *shortest superstring* and *most overlap* are one objective.

The overlap graph

nodes = reads edges = overlaps, weighted by $ov(\cdot, \cdot)$
(only $ov \geq 2$ drawn, to stay legible)



A superstring using every read once = a path visiting every node once.

AAA → AAB → ABB
→ BBB → BBA

spells AAABBBBA — the SCS from the definition slide.

Maximising total overlap = minimising superstring length. Negate the weights and it is *Traveling Salesman*; drop them and it is *Hamiltonian Path*. This is the object the next slides — and slide 51 — actually operate on.

Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

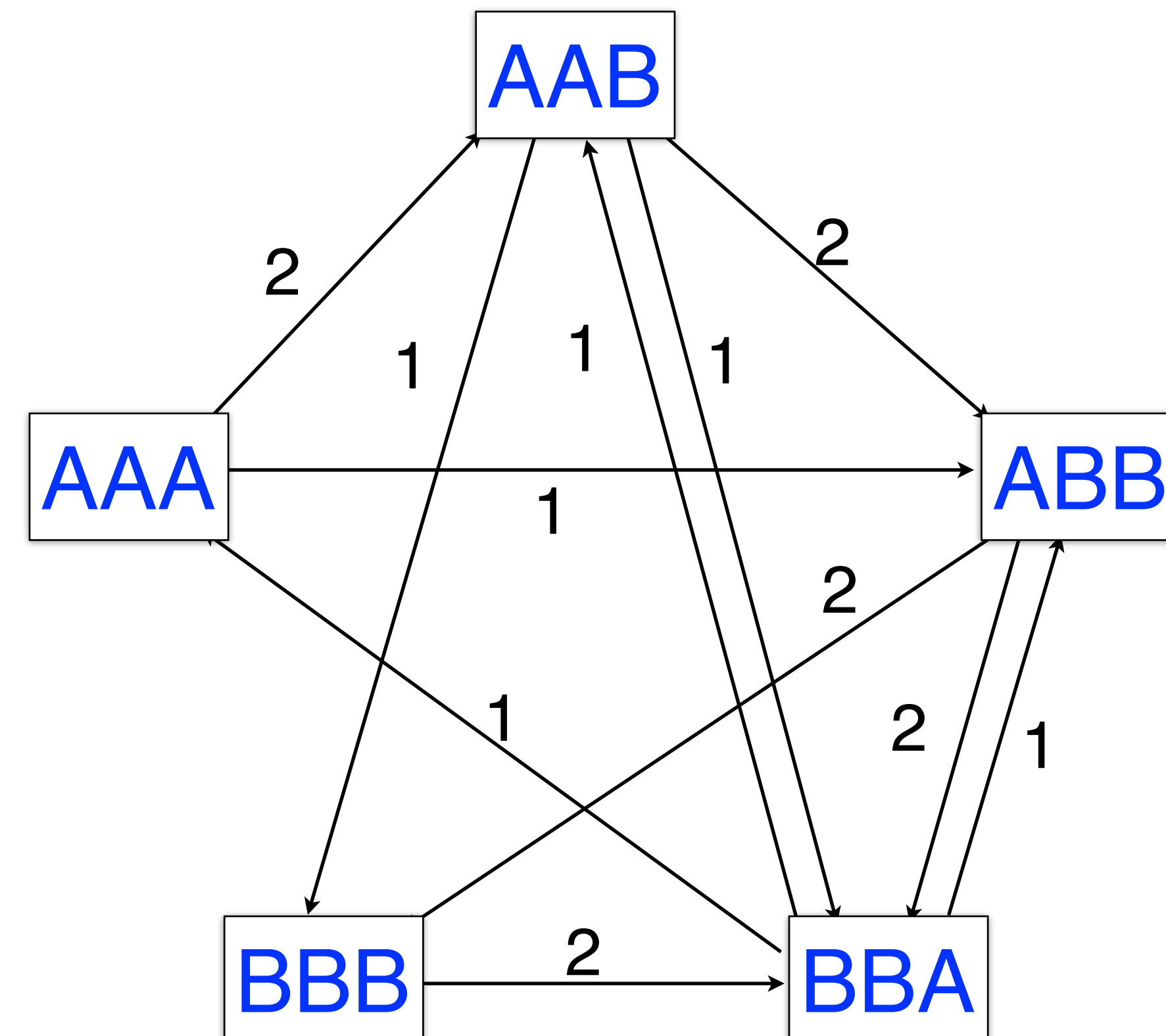
┌─── Input strings ──┐
AAA AAB ABB BBB BBA

Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

┌─── Input strings ──┐
AAA AAB ABB BBB BBA

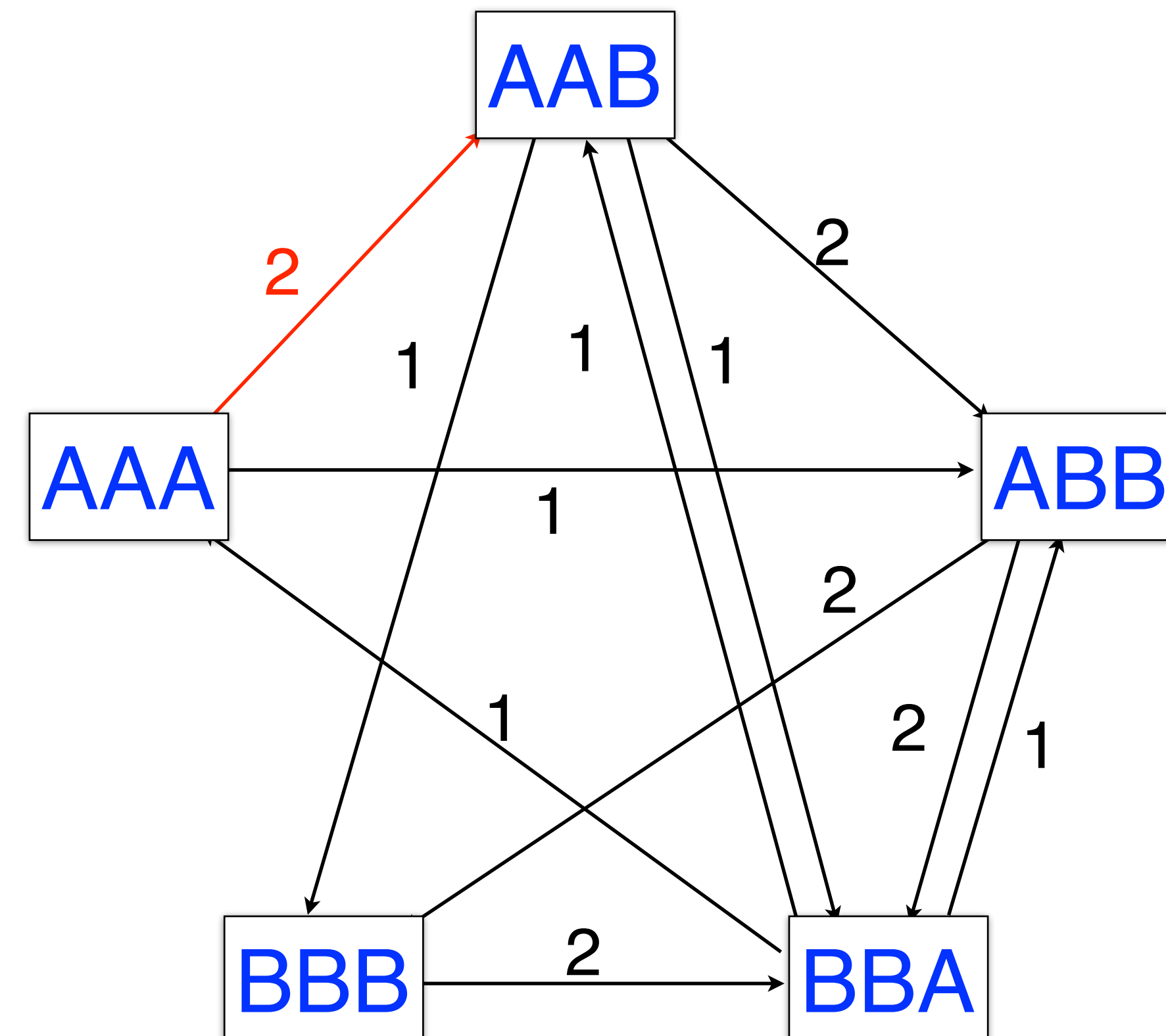


Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

┌─── Input strings ──┐
AAA AAB ABB BBB BBA
AAA AAB ABB BBB BBA



Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

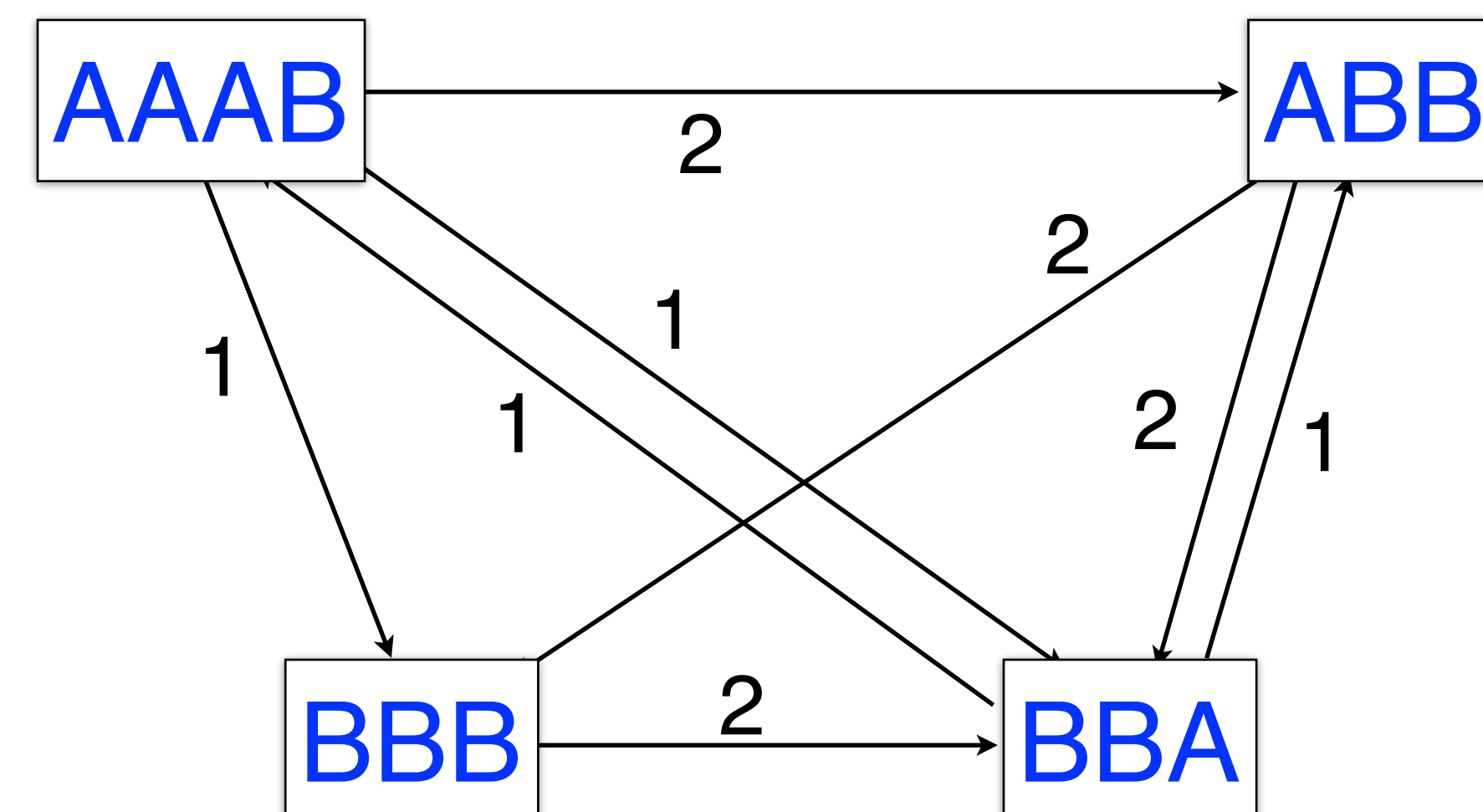
Algorithm in action ($l = 1$):

┌─── Input strings ──┐

AAA AAB ABB BBB BBA

AAA AAB ABB BBB BBA

AAAB ABB BBB BBA



Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

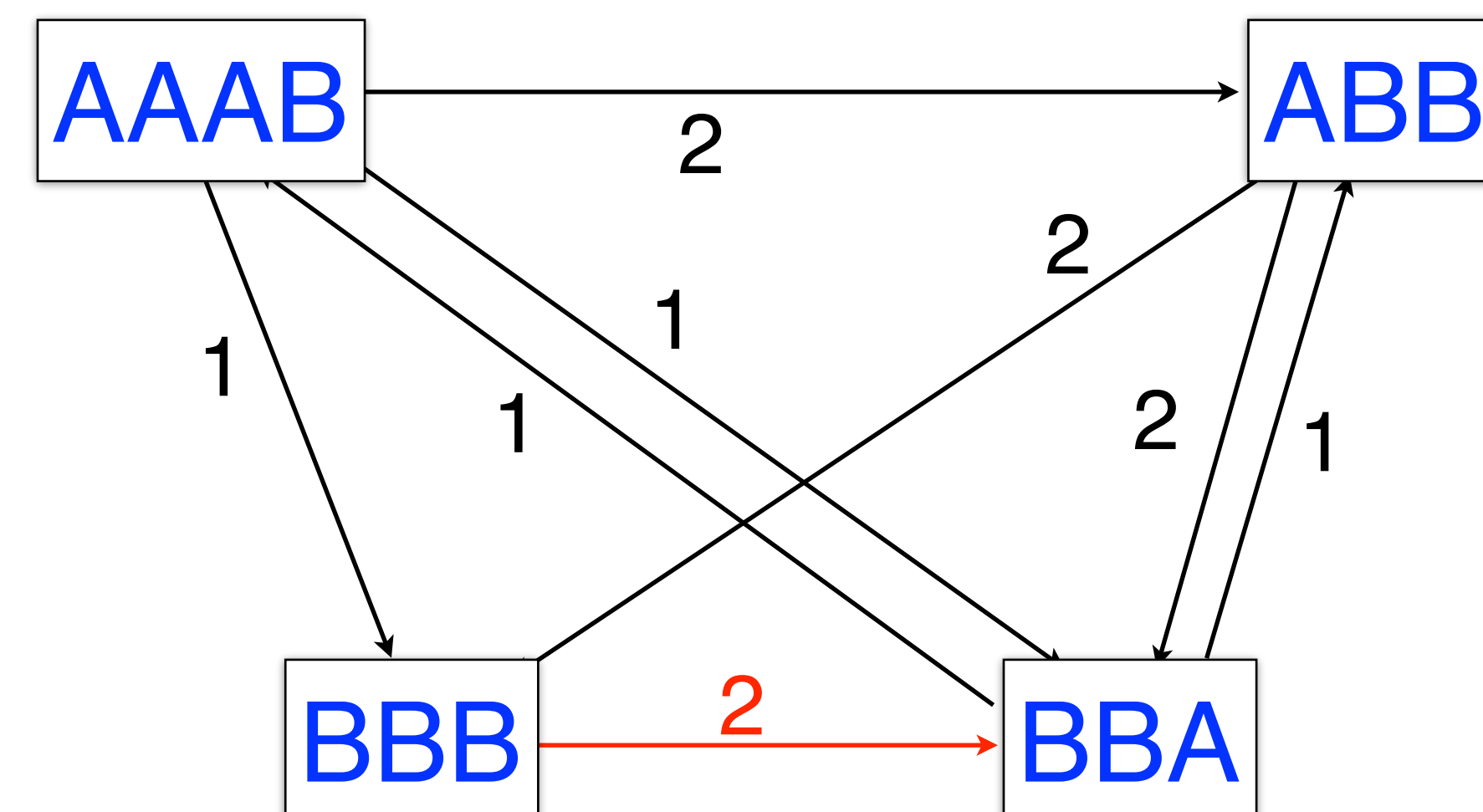
Algorithm in action ($l = 1$):

┌─── Input strings ──┐

AAA AAB ABB BBB BBA

AAA AAB ABB BBB BBA

AAAB ABB BBB BBA



Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

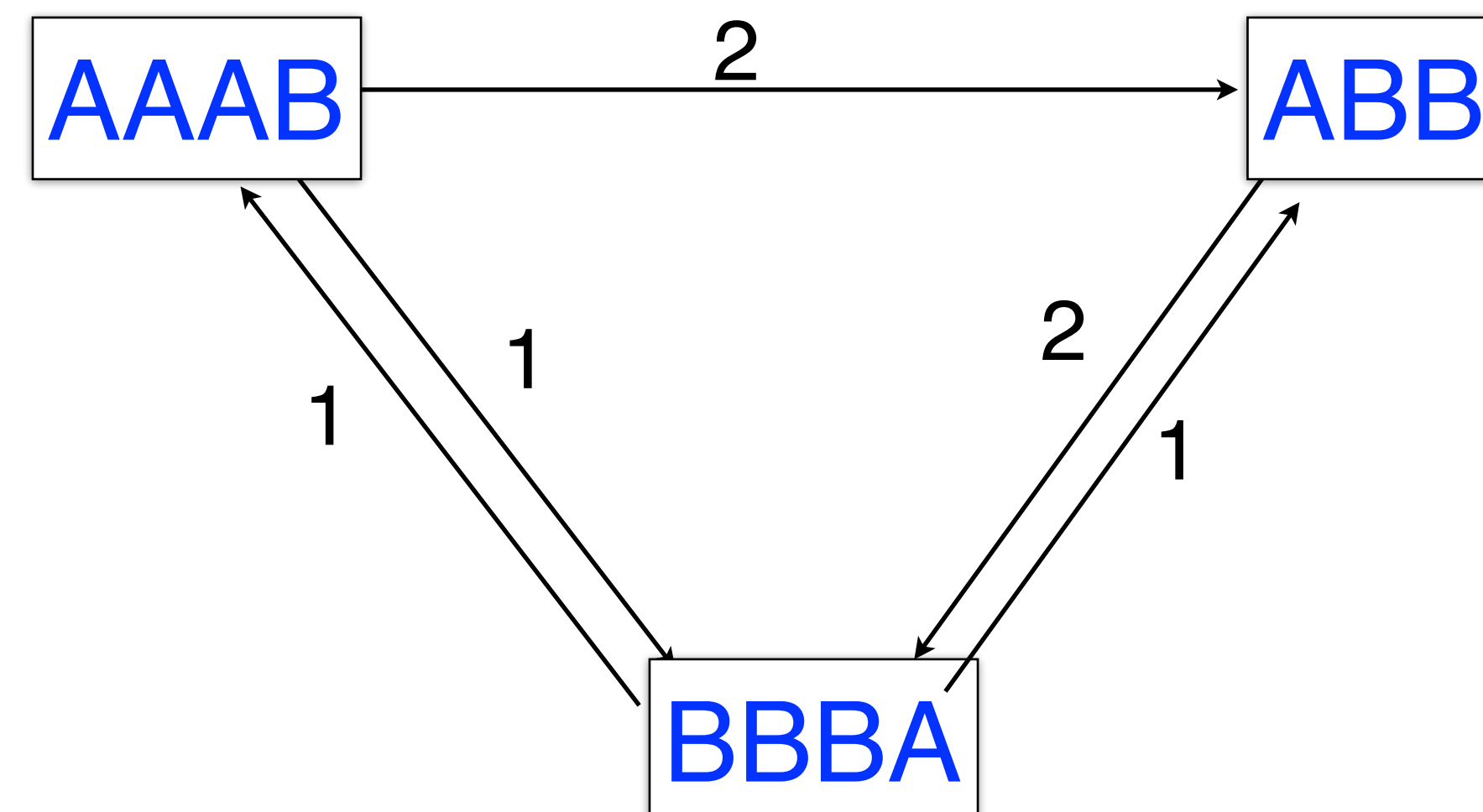
┌─── Input strings ──┐

AAA AAB ABB BBB BBA

AAA AAB ABB BBB BBA

AAAB ABB BBB BBA

AAAB BBBA ABB



Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

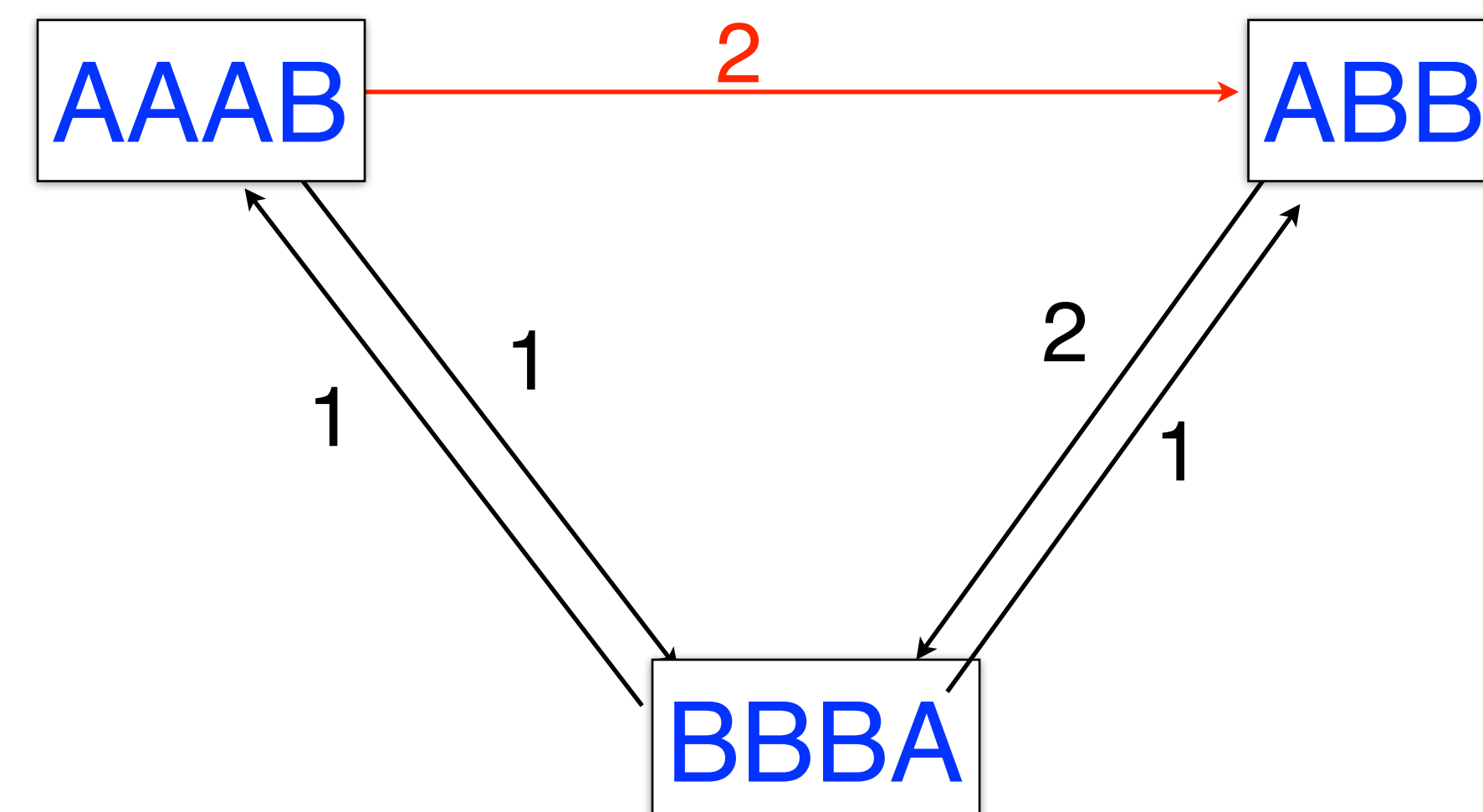
┌─── Input strings ──┐

AAA AAB ABB BBB BBA

AAA AAB ABB BBB BBA

AAAB ABB BBB BBA

AAAB BBBA ABB



Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

┌─── Input strings ──┐

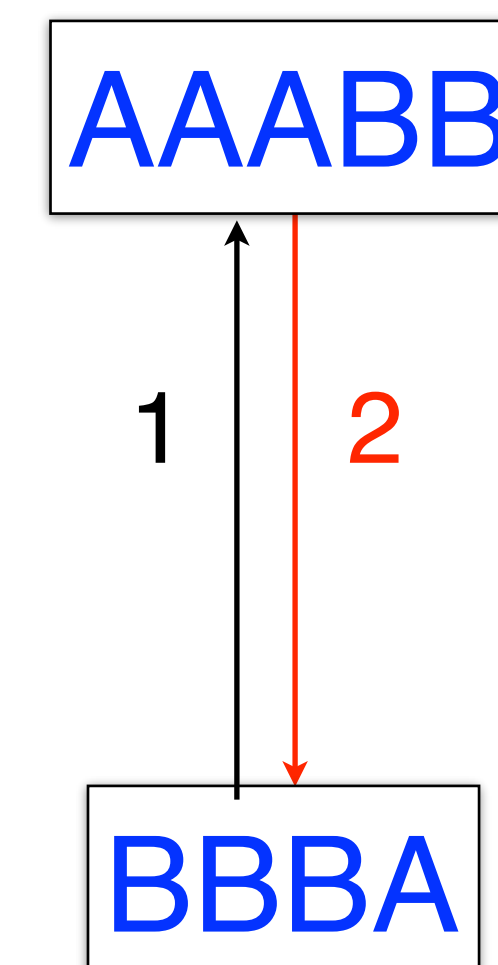
AAA AAB ABB BBB BBA

AAA AAB ABB BBB BBA

AAAB ABB BBB BBA

AAAB BBBA ABB

AAABB BBBA



Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

┌─── Input strings ──┐

AAA AAB ABB BBB BBA

AAA AAB ABB BBB BBA

AAAB ABB BBB BBA

AAAB BBBA ABB

AAABB BBBA

AAABBBA

AAABBBA

Shortest common superstring: greedy

Greedy-SCS: in each round, merge pair of strings with maximal overlap. Stop when no more overlaps exist. Concatenate resulting strings. l = minimum overlap.

Algorithm in action ($l = 1$):

┌─── Input strings ──┐

AAA AAB ABB BBB BBA

AAA AAB ABB BBB BBA

AAAB ABB BBB BBA

AAAB BBBA ABB

AAABB BBBA

AAABBBA

AAABBBA

That's the SCS

Greedy shortest common superstring (when is it not optimal?)

AAA AAB ABB BBA BBB
↓ ↙
AAAB ABB BBA BBB

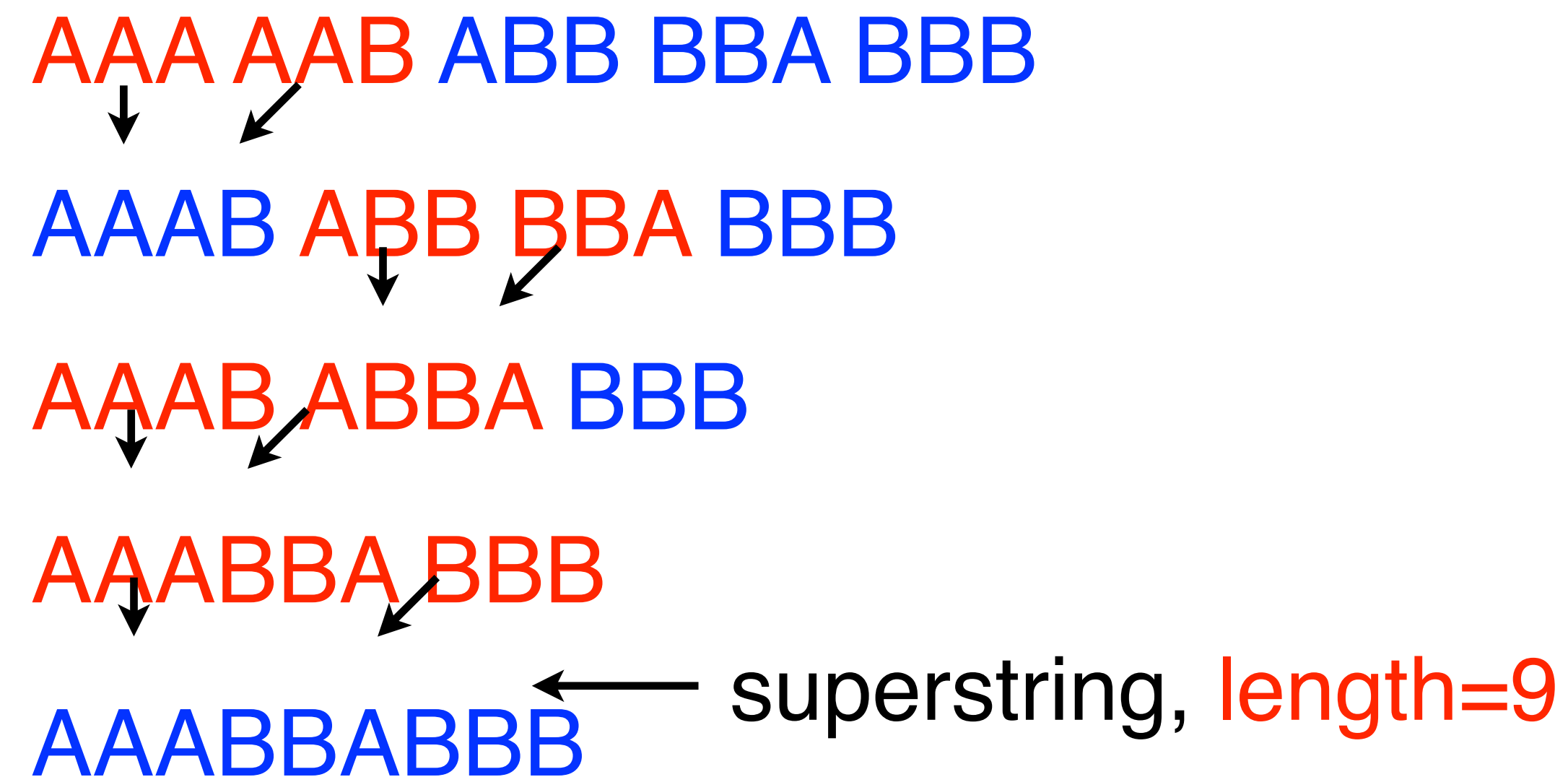
Greedy shortest common superstring (when is it not optimal?)

AAA AAB ABB BBA BBB
↓ ↘
AAAB ABB BBA BBB
↓ ↘
AAAB ABBA BBB

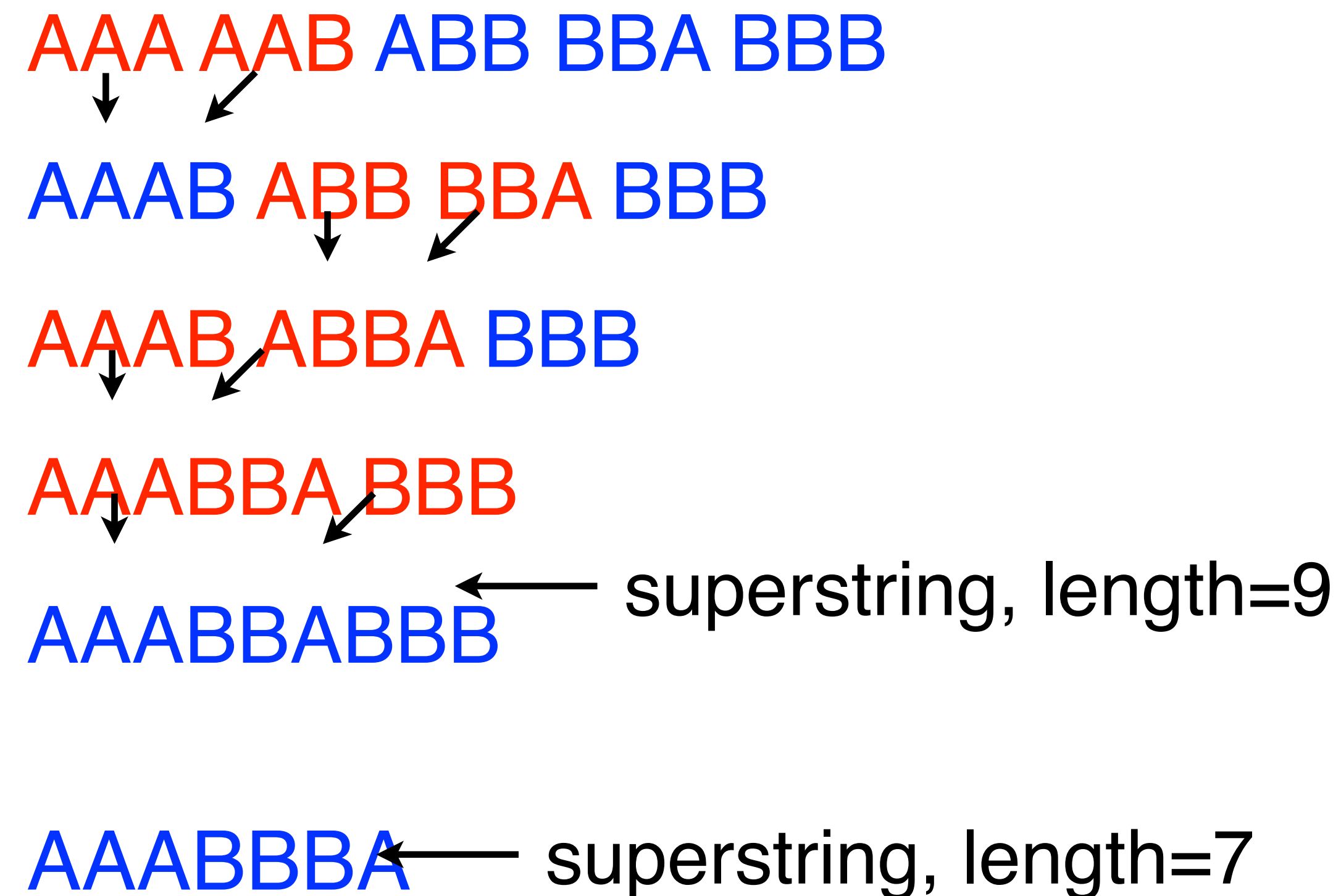
Greedy shortest common superstring (when is it not optimal?)

AAA AAB ABB BBA BBB
↓ ↘
AAAB ABB BBA BBB
↓ ↘
AAAB ABBA BBB
↓ ↘
AAABBA BBB

Greedy shortest common superstring (when is it not optimal?)



Greedy shortest common superstring (when is it not optimal?)



Greedy answer *isn't necessarily optimal*

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
[a_long_long_long_time](#). $l = 3$.

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long **g_time ng_tim**

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
a_long_long_long_time. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long **g_time ng_tim**
ng_time ng_lon **_long_** a_long long_l ong_ti ong_lo long_t **g_long**

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long **g_time ng_tim**
ng_time ng_lon **_long_** a_long long_l ong_ti ong_lo long_t **g_long**
ng_time g_long_ ng_lon a_long long_l **ong_ti** ong_lo **long_t**

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
ng_time long_ti g_long_ ng_lon a_long long_l ong_lo

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
ng_time long_ti g_long_ ng_lon a_long long_l ong_lo
ng_time ong_lon long_ti g_long_ a_long long_l

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
ng_time long_ti g_long_ ng_lon a_long long_l ong_lo
ng_time ong_lon long_ti g_long_ a_long long_l
ong_lon long_time g_long_ a_long long_l

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
ng_time long_ti g_long_ ng_lon a_long long_l ong_lo
ng_time ong_lon long_ti g_long_ a_long long_l
ong_lon long_time g_long_ a_long long_l
long_lon long_time g_long_ a_long

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
ng_time long_ti g_long_ ng_lon a_long long_l ong_lo
ng_time ong_lon long_ti g_long_ a_long long_l
ong_lon long_time g_long_ a_long long_l
long_lon long_time g_long_ a_long
long_lon g_long_time a_long

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
ng_time long_ti g_long_ ng_lon a_long long_l ong_lo
ng_time ong_lon long_ti g_long_ a_long long_l
ong_lon long_time g_long_ a_long long_l
long_lon long_time g_long_ a_long
long_lon g_long_time a_long
long_long_time a_long

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
ng_time long_ti g_long_ ng_lon a_long long_l ong_lo
ng_time ong_lon long_ti g_long_ a_long long_l
ong_lon long_time g_long_ a_long long_l
long_lon long_time g_long_ a_long
long_lon g_long_time a_long
long_long_time a_long
a_long_long_time

Shortest common superstring: greedy

Why else might it not be a good model for assembly?

Greedy-SCS assembling all substrings of length 6 from:
`a_long_long_time`. $l = 3$.

ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long g_time ng_tim
ng_time ng_lon _long_ a_long long_l ong_ti ong_lo long_t g_long
ng_time g_long_ ng_lon a_long long_l ong_ti ong_lo long_t
ng_time long_ti g_long_ ng_lon a_long long_l ong_lo
ng_time ong_lon long_ti g_long_ a_long long_l
ong_lon long_time g_long_ a_long long_l
long_lon long_time g_long_ a_long
long_lon g_long_time a_long
long_long_time a_long
a_long_long_time



Foiled by repeat!

Shortest common superstring: greedy

Same example, but increased the substring length from 6 to 8

long_lon ng_long_ _long_lo g_long_t ong_long g_long_l ong_time a_long_l _long_ti long_tim
long_time long_lon ng_long_ _long_lo g_long_t ong_long g_long_l a_long_l _long_ti
_long_time long_lon ng_long_ _long_lo g_long_t ong_long g_long_l a_long_l
_long_time a_long_lo long_lon ng_long_ g_long_t ong_long g_long_l
_long_time ong_long_ a_long_lo long_lon g_long_t g_long_l
g_long_time ong_long_ a_long_lo long_lon g_long_l
g_long_time ong_long_ a_long_lon g_long_l
g_long_time ong_long_l a_long_lon
g_long_time a_long_long_l
a_long_long_long_time
a_long_long_long_time

Got the whole thing: a_long_long_long_time

Shortest common superstring: greedy

Why are substrings of length 8 long enough for Greedy-SCS to figure out there are 3 copies of **long**?

a_long_long_long_time

Shortest common superstring: greedy

Why are substrings of length 8 long enough for Greedy-SCS to figure out there are 3 copies of **long**?

a_long_long_long_time

g_long_l

Shortest common superstring: greedy

Why are substrings of length 8 long enough for Greedy-SCS to figure out there are 3 copies of **long**?

a_long_long_long_time

g_long_l



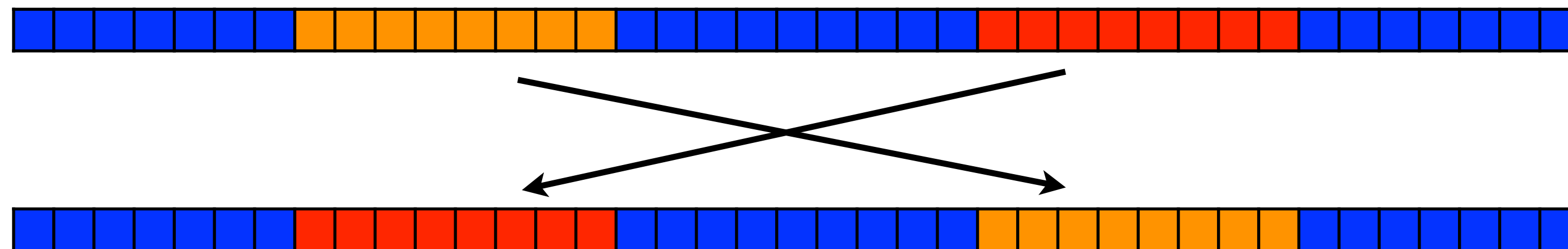
One length-8 substring spans all three **longs**

Third law of assembly

Repeats make assembly difficult; whether we can assemble without mistakes depends on length of reads and repetitive patterns in genome

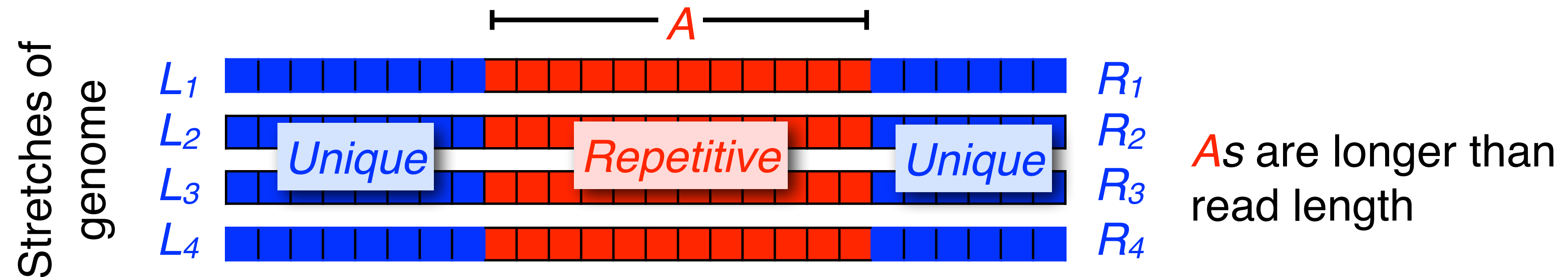
Collapsing a *tandem* repeat: **a_long_long_long_time**
↓
a_long_long_time

Spurious rearrangement:



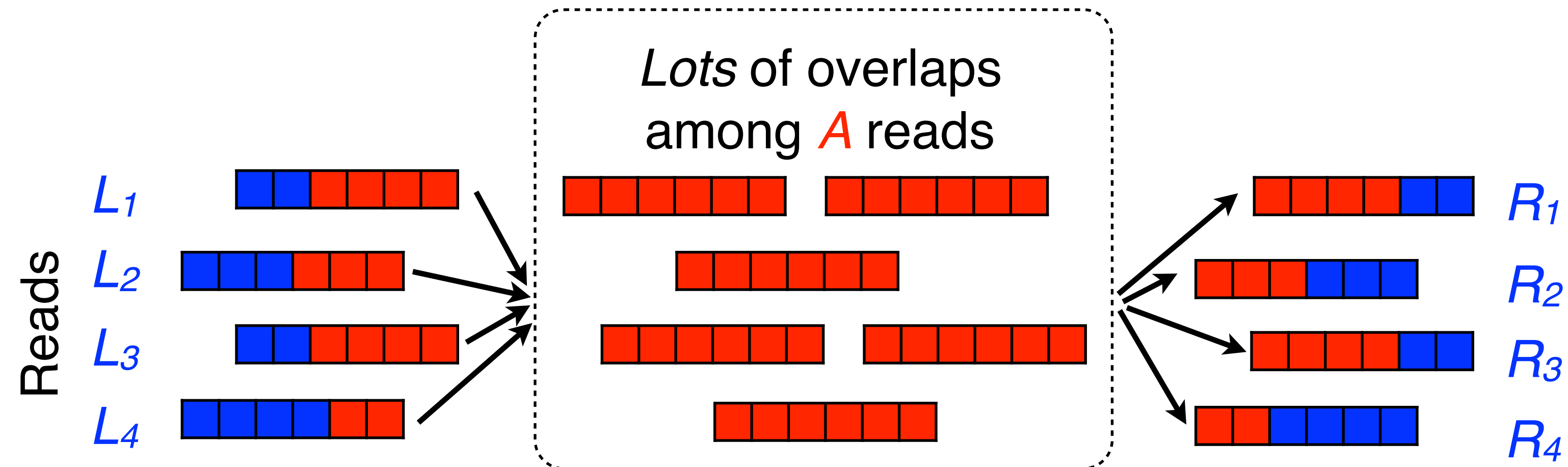
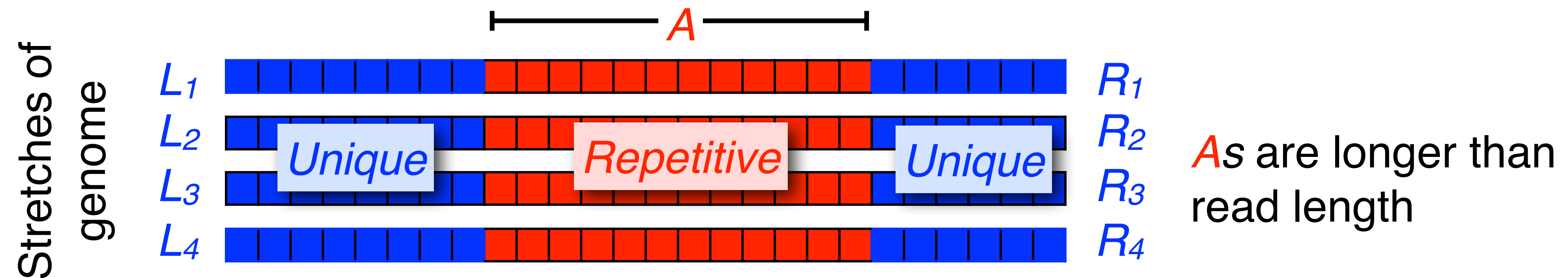
Repeats foil assembly

Portion of overlap graph involving repeat family *A*



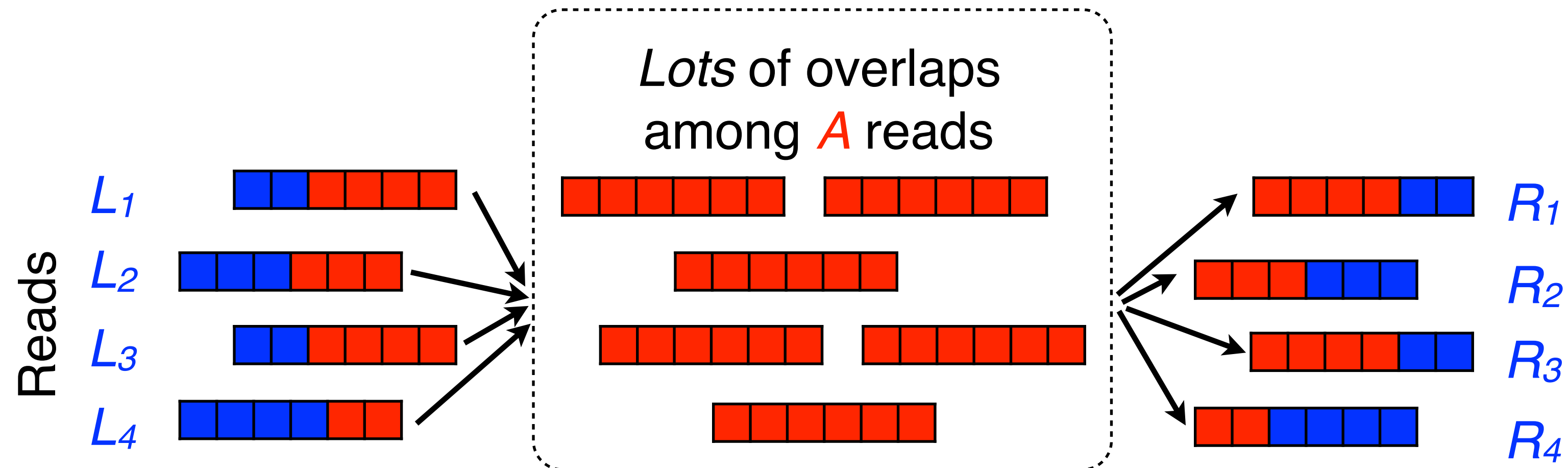
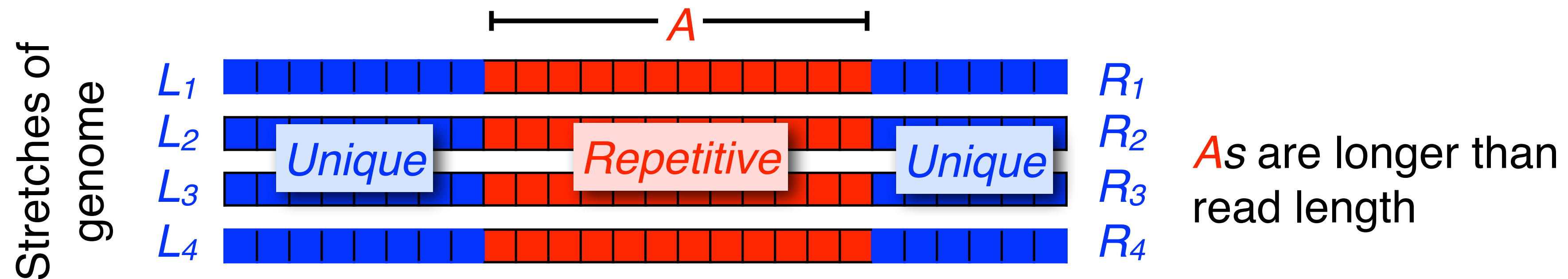
Repeats foil assembly

Portion of overlap graph involving repeat family *A*



Repeats foil assembly

Portion of overlap graph involving repeat family *A*



Even if we avoid collapsing copies of *A*, we can't know which paths *in* correspond to which paths *out*

A practical implementation of SCS

Two components of the SCS implementation make things slow from a practical perspective on large data

1) All vs. all string comparison makes computing the initial edge set slow.

In practice, this is often solved by requiring minimal overlap thresholds between joined strings, and using a hash table to turn all-pairs comparison into lookup.

2) Searching / scanning for the highest-scoring edge in the overlap graph requires $O(E)$ work in each iteration

In practice, this can be solved with a combination of an efficient priority queue and a union-find data structure to keep track of merged strings.

Take-home message:

We are interested in *correct and efficient algorithms* for solving *well-specified* problems.

We must be careful about how we *pose* the problems.

Actually, shortest common superstring is a rather poor model for sequence assembly, due to repeats and errors.